

None

Near Infrared and Hyperspectral Imaging in tandem with Machine Learning for Fraud Detection in Coffee

1. Research Outlook

Food fraud is an increasingly pressing concern, affecting both consumer confidence and the integrity of global markets. Coffee, one of the world's most consumed beverages, is particularly vulnerable to adulteration, where cheaper, inferior substances are blended with authentic products. This project will explore the use of Near-Infrared Hyperspectral Imaging (NIR-HSI) combined with Machine Learning (ML) techniques to detect coffee adulteration. NIR-HSI captures detailed spectral and spatial data across a wide range of wavelengths, revealing subtle chemical and physical differences. The aim is to explore the potential of combining this technology with chemometrics and machine learning to detect fraud in both ground and freeze-dried instant Arabica coffee adulterated with Robusta.

The collected data will undergo preprocessing, followed by Principal Component Analysis (PCA) for dimensionality reduction. Supervised and unsupervised classification techniques will then be employed to classify samples based on their authenticity, while regression models will be used to quantify the degree of adulteration. This combined approach seeks to develop a reliable, scientific framework for detecting fraud in processed coffee, contributing to improved transparency and quality assurance in the coffee industry.

2. Data

A total of 30 coffee samples were purchased from different countries, all of which are major coffee producers. This included 25 Arabica and 5 Robusta samples. The beans were roasted to a medium roast for 15 minutes at 220°C. After roasting, the coffee was cooled overnight and then ground into a fine powder for analysis.

The adulteration study involved mixing 16 Arabica samples with 3 distinct Robusta samples to create the training set, while the remaining 9 Arabica samples were mixed with 2 Robusta samples to form the test set. Adulteration levels were set at 0%, 1%, 5%, 10%, 20%, 40%, 60%, 80%, and 100% to simulate different levels of fraud. The samples were subsequently analysed by Hyperspectral Imaging (HSI) in triplicate.

3. Data Cleaning

```
In [1]: # Load packages
import os
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
```

```
In [2]: # Set the working directory
```

```
# Change working directory
os.chdir('C:\\Users\\abc\\OneDrive - UGent\\Documenten\\Derick Malavi_PhD Docs_UGent\\Manuscript 4_Coffee\\Coffee_Research_Trajectory.html')
```

```
In [3]: # Load the hyperspectral imaging data
data = 'gr_coffee_compiled_final.csv'
coffee_data = pd.read_csv(data)
```

```
In [4]: # Check the number of rows and columns
coffee_data.shape
```

```
Out[4]: (1469, 229)
```

```
In [5]: # Check whether it is a data frame
type(coffee_data)
```

```
Out[5]: pandas.core.frame.DataFrame
```

```
In [6]: #Check the first five rows of the data
coffee_data.head(10)
```

```
Out[6]:
```

	sample_id	binary_class	three_class	adult_percent	cal_val	935.609985	939.059998	942.059998
0	Pure_Arabica_10	pure_arabica	pure_arabica	0	1	0.347876	0.349376	0.350876
1	Pure_Arabica_10	pure_arabica	pure_arabica	0	1	0.354493	0.356088	0.357683
2	Pure_Arabica_10	pure_arabica	pure_arabica	0	1	0.354535	0.355939	0.357343
3	Pure_Arabica_11	pure_arabica	pure_arabica	0	1	0.385204	0.386921	0.388638
4	Pure_Arabica_11	pure_arabica	pure_arabica	0	1	0.386532	0.388493	0.390454
5	Pure_Arabica_11	pure_arabica	pure_arabica	0	1	0.379677	0.381715	0.383753
6	Pure_Arabica_12	pure_arabica	pure_arabica	0	1	0.435689	0.438137	0.440585
7	Pure_Arabica_12	pure_arabica	pure_arabica	0	1	0.440689	0.443175	0.445661
8	Pure_Arabica_12	pure_arabica	pure_arabica	0	1	0.437546	0.440136	0.442726
9	Pure_Arabica_13	pure_arabica	pure_arabica	0	1	0.403580	0.405651	0.407722

10 rows × 229 columns

```
In [7]: # Check for any missing values
coffee_data.isnull().sum().any()
```

```
Out[7]: False
```

```
In [8]: # Sort/Arrange the data frame based on the percent adulteration column

coffee_data = coffee_data.sort_values(by = ['adult_percent'], ascending=True)
coffee_data_save = coffee_data # To be saved for further analysis
```

```
In [9]: # The values have to be between 0 and 1. Let us check if there any extreme values to be removed

min_value = coffee_data.iloc[:, 5:226].min().min() # Overall min value
max_value = coffee_data.iloc[:, 5:226].max().max() # Overall max value

print(f'The minimum value is {min_value} and the maximum value is {max_value}')
```

The minimum value is 0.248572 and the maximum value is 0.737226

In [10]: `coffee_data.head(10)`

Out[10]:

	sample_id	binary_class	three_class	adult_percent	cal_val	935.609985	939.059998	941	
	0	Pure_Arabica_10	pure_arabica	pure_arabica	0	1	0.347876	0.349376	C
	54	Pure_Arabica_3	pure_arabica	pure_arabica	0	1	0.332850	0.334093	C
	53	Pure_Arabica_2	pure_arabica	pure_arabica	0	1	0.347639	0.349199	C
	52	Pure_Arabica_2	pure_arabica	pure_arabica	0	1	0.338413	0.339599	C
	51	Pure_Arabica_2	pure_arabica	pure_arabica	0	1	0.343888	0.345065	C
	50	Pure_Arabica_25	pure_arabica	pure_arabica	0	2	0.334076	0.335419	C
	49	Pure_Arabica_25	pure_arabica	pure_arabica	0	2	0.336468	0.337828	C
	48	Pure_Arabica_25	pure_arabica	pure_arabica	0	2	0.334080	0.335349	C
	47	Pure_Arabica_24	pure_arabica	pure_arabica	0	2	0.343569	0.344847	C
	46	Pure_Arabica_24	pure_arabica	pure_arabica	0	2	0.342920	0.344168	C

10 rows × 229 columns

In [11]: `# Check the number of samples for the classes`

```
binary_class = coffee_data.groupby('binary_class')['binary_class'].count()
print(binary_class)
```

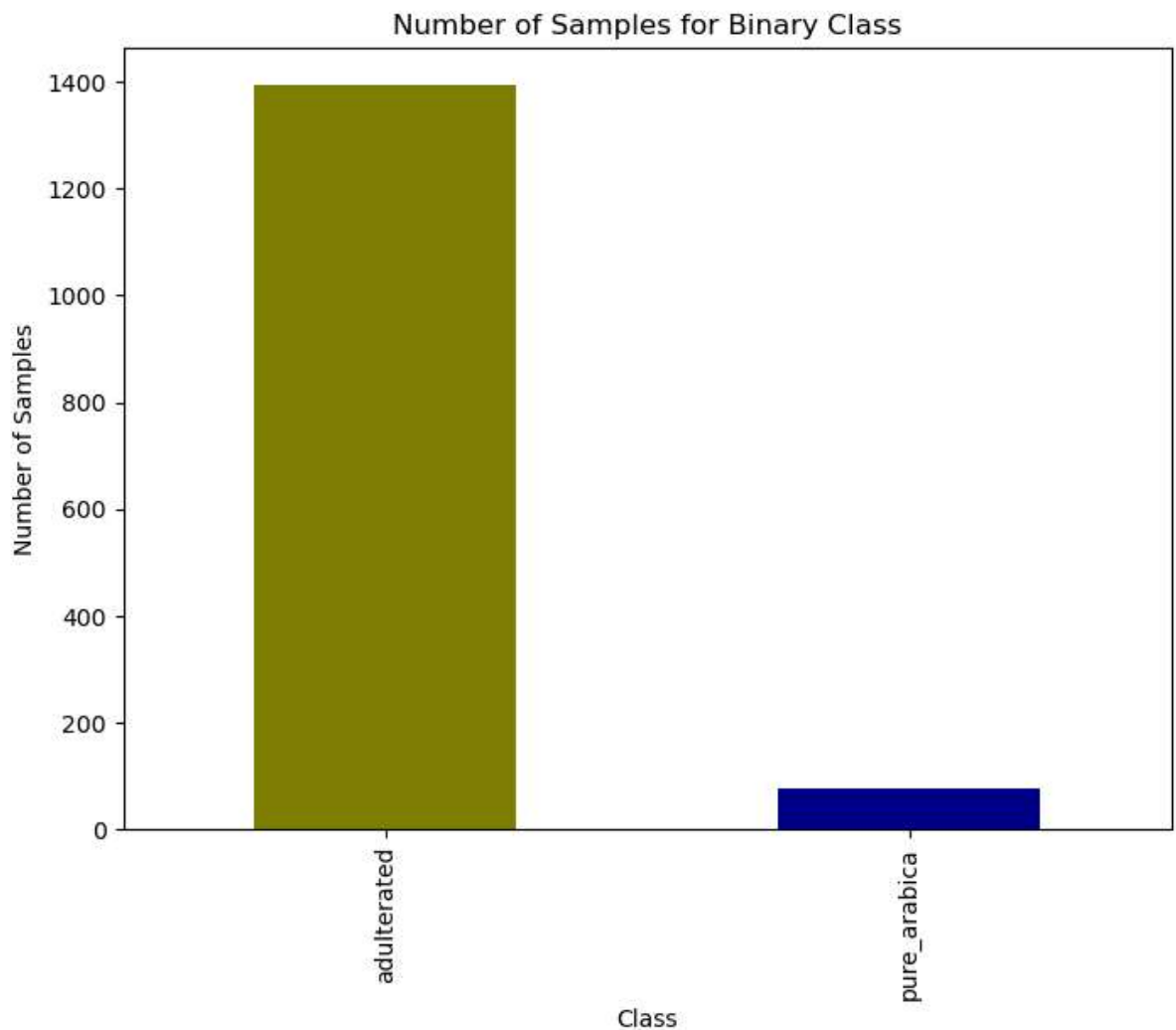
```
binary_class
adulterated      1394
pure_arabica       75
Name: binary_class, dtype: int64
```

In [12]: `# Plotting bar graph`

```
plt.figure(figsize=(8,6))
binary_class.plot(kind='bar', color=['olive', 'darkblue'])

# Add Labels and title
plt.title('Number of Samples for Binary Class', fontsize=12, loc='center')
plt.xlabel('Class', fontsize=10, loc='center')
plt.ylabel('Number of Samples', fontsize=10, loc='center')

# Display the plot
plt.show()
```



- The two classes are highly imbalanced. We will have to balance the dataset when training the classification models. Techniques such as SMOTE will be evaluated.

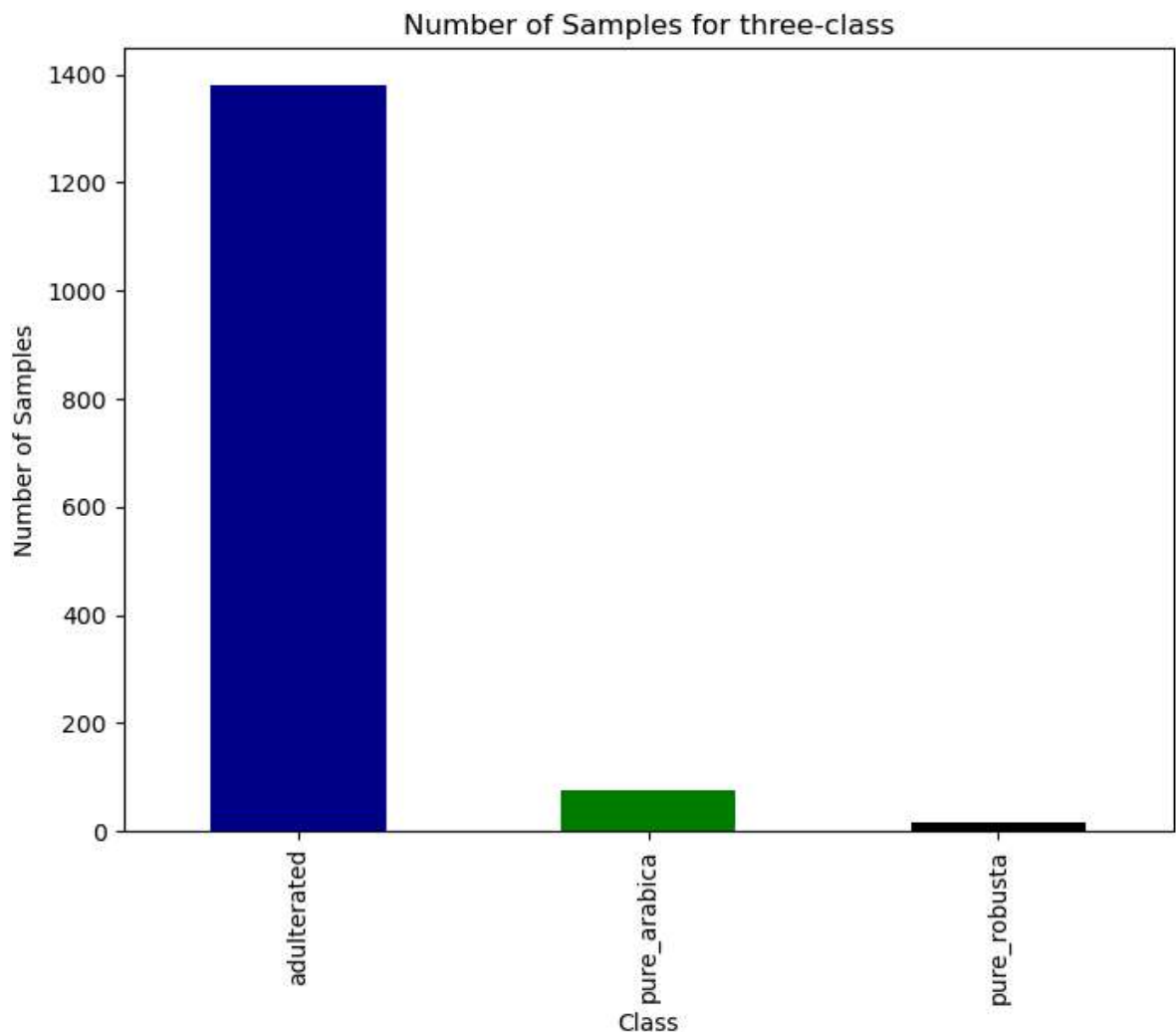
```
In [13]: # Let us also check the three classes
three_class = coffee_data.groupby('three_class')['three_class'].count()
print(three_class)
```

```
three_class
adulterated      1379
pure_arabica       75
pure_robusta      15
Name: three_class, dtype: int64
```

```
In [14]: # Plotting
plt.figure(figsize=(8,6))
three_class.plot(kind='bar', color=['darkblue', 'green', 'black'])

# Add Labels and title
plt.title('Number of Samples for three-class', fontsize=12, loc='center')
plt.xlabel('Class', fontsize=10, loc='center')
plt.ylabel('Number of Samples', fontsize=10, loc='center')

# Display the plot
plt.show()
```



```
In [15]: # Check the number of samples for training/cross-validation and testing models
calibration = coffee_data[coffee_data['cal_val'] == 1]['cal_val'].count()
external_validation = coffee_data[coffee_data['cal_val'] == 2]['cal_val'].count()
percent_calibration = round((calibration/(calibration + external_validation))*100,2)
percent_validation = round((external_validation/(calibration + external_validation))*100,2)

print(f'The number of samples for training models will be {calibration} samples or {percent_calibration}% and the
The number of samples for training models will be 1055 samples or 71.82%, while the
use for testing the models will be 414 samples or 28.18%.
```

```
In [16]: spectral_columns = coffee_data.iloc[:,5:226]
spectral_columns.head()
```

```
Out[16]:
```

	935.609985	939.059998	942.52002	945.97998	949.429993	952.890015	956.349976	959.80
0	0.347876	0.349376	0.363904	0.375547	0.384829	0.392920	0.398859	0.40
54	0.332850	0.334093	0.346032	0.356395	0.364701	0.372712	0.378541	0.38
53	0.347639	0.349199	0.363783	0.375387	0.384600	0.393099	0.399060	0.40
52	0.338413	0.339599	0.350815	0.362448	0.370806	0.378820	0.384661	0.38
51	0.343888	0.345065	0.357601	0.368907	0.377726	0.385786	0.391651	0.39

5 rows × 221 columns

4. Plotting the Spectra

```
In [17]: # It is critical to visualize the spectra. We can first plot 'blindly' without the
# Get the wavelengths from the column names

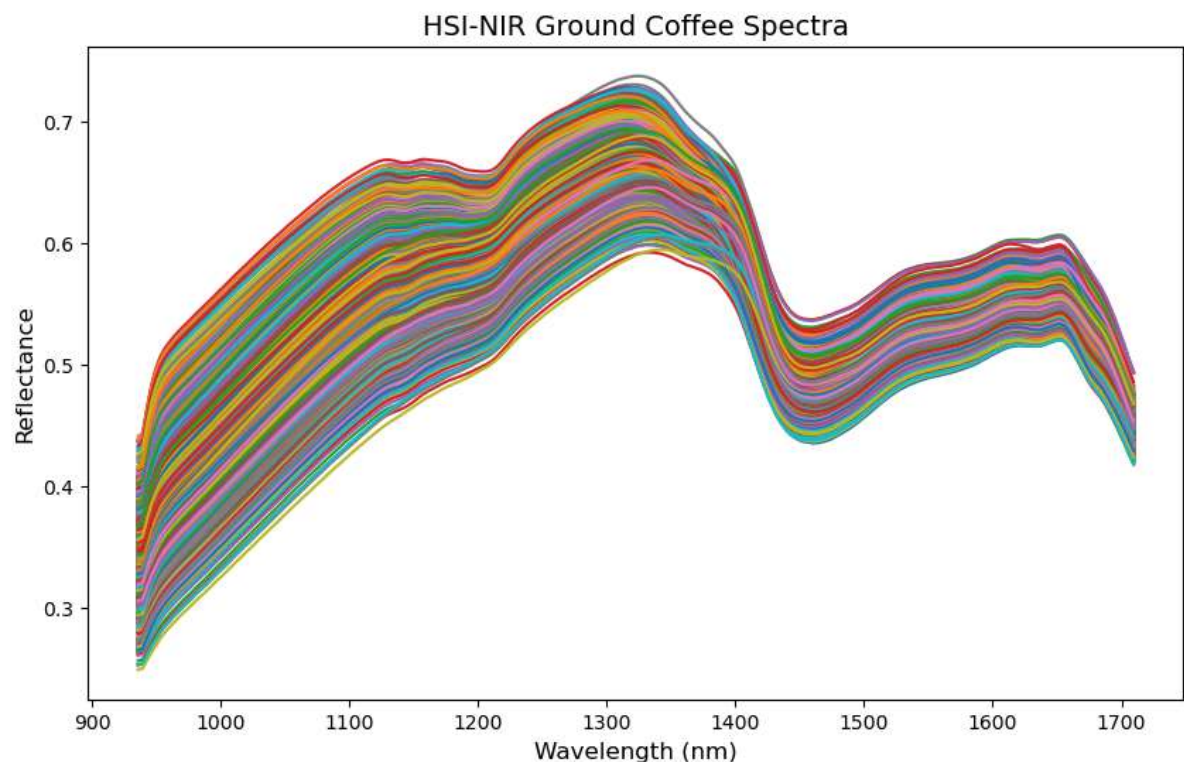
wavelengths = spectral_columns.columns.astype(float)

# Plot the spectral data
plt.figure(figsize=(10, 6))

# Iterate through each row in the spectral data and plot the spectrum
for index, row in spectral_columns.iterrows():
    plt.plot(wavelengths, row, label=f'Sample {index}')

# Add title and Labels
plt.title('HSI-NIR Ground Coffee Spectra', fontsize=14)
plt.xlabel('Wavelength (nm)', fontsize=12)
plt.ylabel('Reflectance', fontsize=12)

# Show the plot
plt.show()
```



```
In [18]: # Spectra plot based on binary classification: pure vs adulterated coffee

# Define the binary class
binary_class = coffee_data['binary_class']

# Extract the spectral wavelengths
wavelengths = spectral_columns.columns.astype(float)

# Define the color pallete
palette = {"adulterated": "maroon", "pure_arabica": "blue"}

# Plot the spectral data with colors based on the binary_class
plt.figure(figsize=(10, 6))

# Iterate over each row and plot, using the binary_class to color each sample
```



```

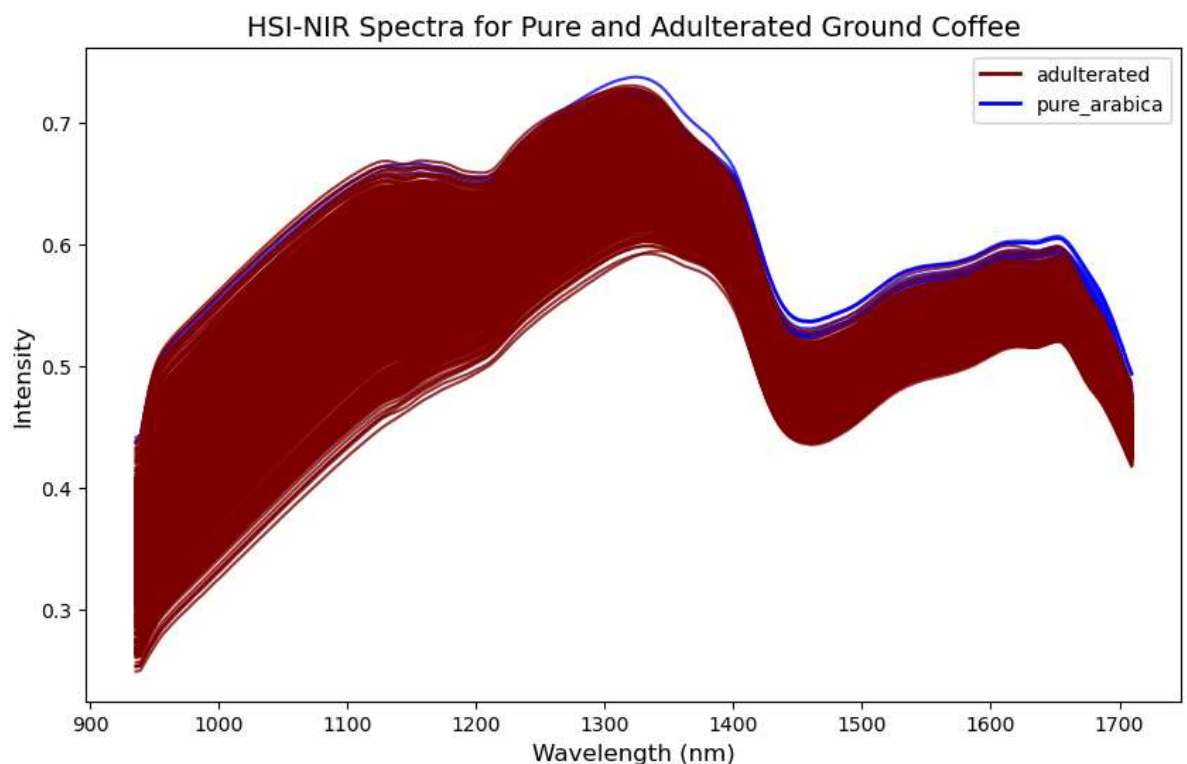
for index, row in spectral_columns.iterrows():
    class_label = binary_class.iloc[index]
    plt.plot(wavelengths, row, color=palette[class_label], label=class_label if index % 10 == 0)

# Add title and Labels
plt.title('HSI-NIR Spectra for Pure and Adulterated Ground Coffee', fontsize=14)
plt.xlabel('Wavelength (nm)', fontsize=12)
plt.ylabel('Intensity', fontsize=12)

# Add Legend
handles = [plt.Line2D([0], [0], color=palette[class_label], lw=2) for class_label in labels]
labels = palette.keys()
plt.legend(handles, labels, loc='upper right')

# Show the plot
plt.show()

```



```

In [19]: # Spectra plot based on three classes: pure arabica, pure robusta and adulterated coffee

# Define the three classes
three_class = coffee_data['three_class']

# Extract the spectral wavelengths
wavelengths = spectral_columns.columns.astype(float)

# Define the color palette
palette = {"adulterated": "grey", "pure_arabica": "darkblue", "pure_robusta": "red"}

# Plot the spectral data with colors based on the three_class
plt.figure(figsize=(10, 6))

# Iterate over each row and plot, using the three_class to color each sample
for index, row in spectral_columns.iterrows():
    class_label = three_class.iloc[index]
    plt.plot(wavelengths, row, color=palette[class_label], label=class_label if index % 10 == 0)

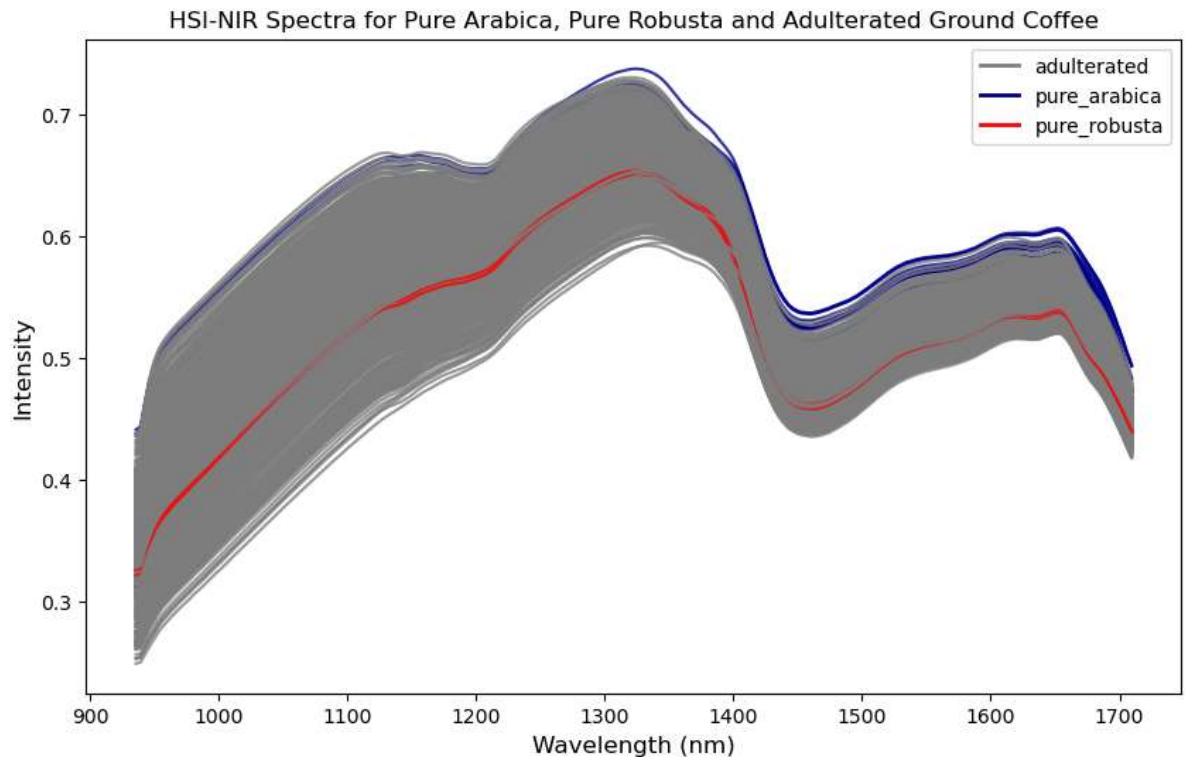
# Add title and Labels
plt.title('HSI-NIR Spectra for Pure Arabica, Pure Robusta and Adulterated Ground Coffee', fontsize=14)
plt.xlabel('Wavelength (nm)', fontsize=12)

```

```
plt.ylabel('Intensity', fontsize=12)

# Add Legend
handles = [plt.Line2D([0], [0], color=palette[class_label], lw=2) for class_label in labels]
labels = palette.keys()
plt.legend(handles, labels, loc='upper right')

# Show the plot
plt.show()
```



```
In [20]: # Save our clean file for further analysis
coffee_data_save.to_csv("coffee_data_pycleaned.csv", index = False)
```

Next analysis

- The data will be preprocessed using several methods and subjected to PCA for dimensionality reduction, followed by both supervised and unsupervised classification. Regression models will also be explored. The research will also explore different variable reduction algorithms and their impact on model performance.